Line Search Methods

Each iteration of a line search method computes a search direction p_k and then decides how far to move along that direction. The iteration is given by

$$x_{k+1} = x_k + \alpha_k p_k, \tag{3.1}$$

where the positive scalar α_k is called the *step length*. The success of a line search method depends on effective choices of both the direction p_k and the step length α_k.

Most line search algorithms require p_k to be a *descent direction*—one for which $p_k^T \nabla f_k < 0$—because this property guarantees that the function f can be reduced along this direction, as discussed in the previous chapter. Moreover, the search direction often has the form

$$p_k = -B_k^{-1} \nabla f_k, \tag{3.2}$$

where B_k is a symmetric and nonsingular matrix. In the steepest descent method B_k is simply the identity matrix I, while in Newton's method B_k is the exact Hessian $\nabla^2 f(x_k)$. In quasi-Newton methods, B_k is an approximation to the Hessian that is updated at every

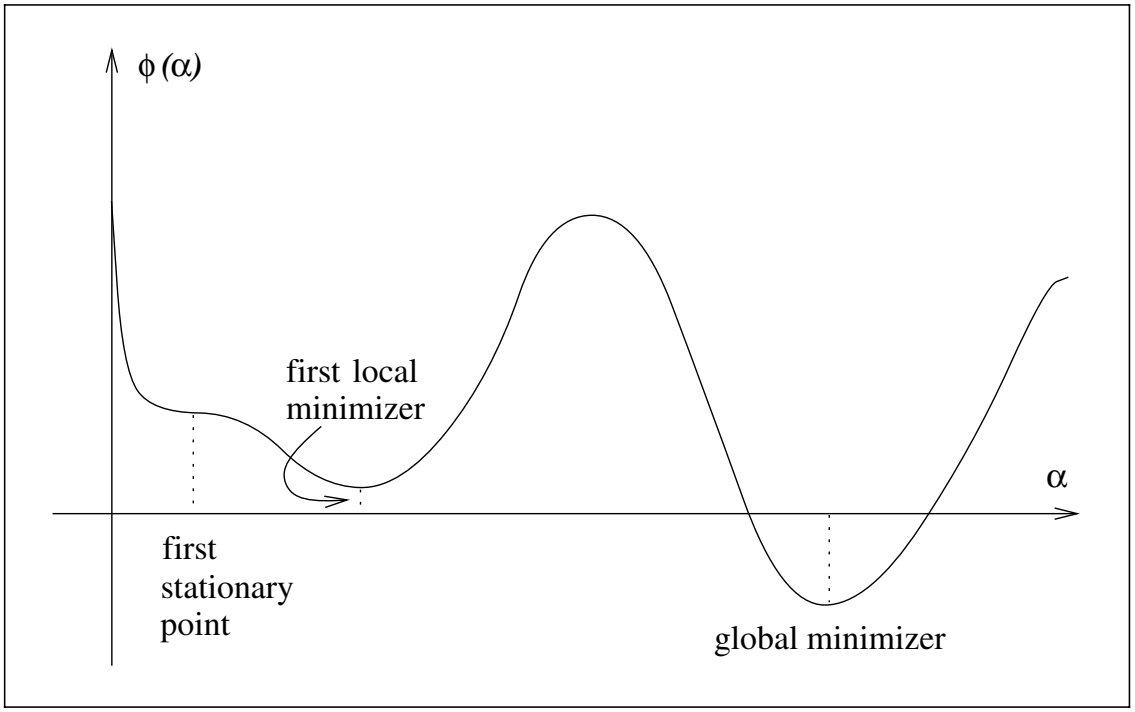


Figure 3.1 The ideal step length is the global minimizer.

iteration by means of a low-rank formula. When p_k is defined by (3.2) and B_k is positive definite, we have

$$p_k^T \nabla f_k = -\nabla f_k^T B_k^{-1} \nabla f_k < 0,$$

and therefore p_k is a descent direction.

In the next chapters we study how to choose the matrix B_k, or more generally, how to compute the search direction. We now give careful consideration to the choice of the step-length parameter α_k.

3.1 STEP LENGTH

In computing the step length α_k, we face a tradeoff. We would like to choose α_k to give a substantial reduction of f, but at the same time, we do not want to spend too much time making the choice. The ideal choice would be the global minimizer of the univariate function $\phi(\cdot)$ defined by

$$\phi(\alpha) = f(x_k + \alpha p_k), \quad \alpha > 0, \tag{3.3}$$

but in general, it is too expensive to identify this value (see Figure 3.1). To find even a local minimizer of ϕ to moderate precision generally requires too many evaluations of the objec-

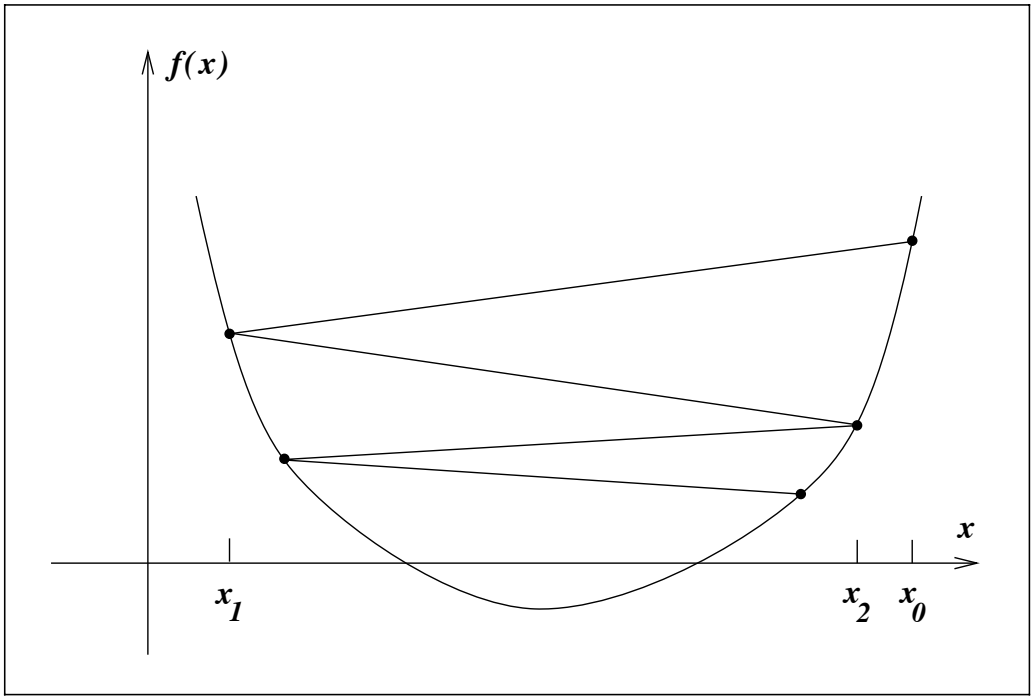


Figure 3.2 Insufficient reduction in f.

tive function f and possibly the gradient ∇f. More practical strategies perform an *inexact* line search to identify a step length that achieves adequate reductions in f at minimal cost.

Typical line search algorithms try out a sequence of candidate values for α, stopping to accept one of these values when certain conditions are satisfied. The line search is done in two stages: A bracketing phase finds an interval containing desirable step lengths, and a bisection or interpolation phase computes a good step length within this interval. Sophisticated line search algorithms can be quite complicated, so we defer a full description until the end of this chapter. We now discuss various termination conditions for the line search algorithm and show that effective step lengths need not lie near minimizers of the univariate function $\phi(\alpha)$ defined in (3.3).

A simple condition we could impose on α_k is that it provide a reduction in f, i.e., $f(x_k + \alpha_k p_k) < f(x_k)$. That this is not appropriate is illustrated in Figure 3.2, where the minimum is $f^* = -1$, but the sequence of function values $\{5/k\}$, $k = 0, 1, \ldots$, converges to zero. The difficulty is that we do not have sufficient reduction in f, a concept we discuss next.

THE WOLFE CONDITIONS

A popular inexact line search condition stipulates that α_k should first of all give *sufficient decrease* in the objective function f, as measured by the following inequality:

$$f(x_k + \alpha p_k) \leq f(x_k) + c_1 \alpha \nabla f_k^T p_k, \tag{3.4}$$

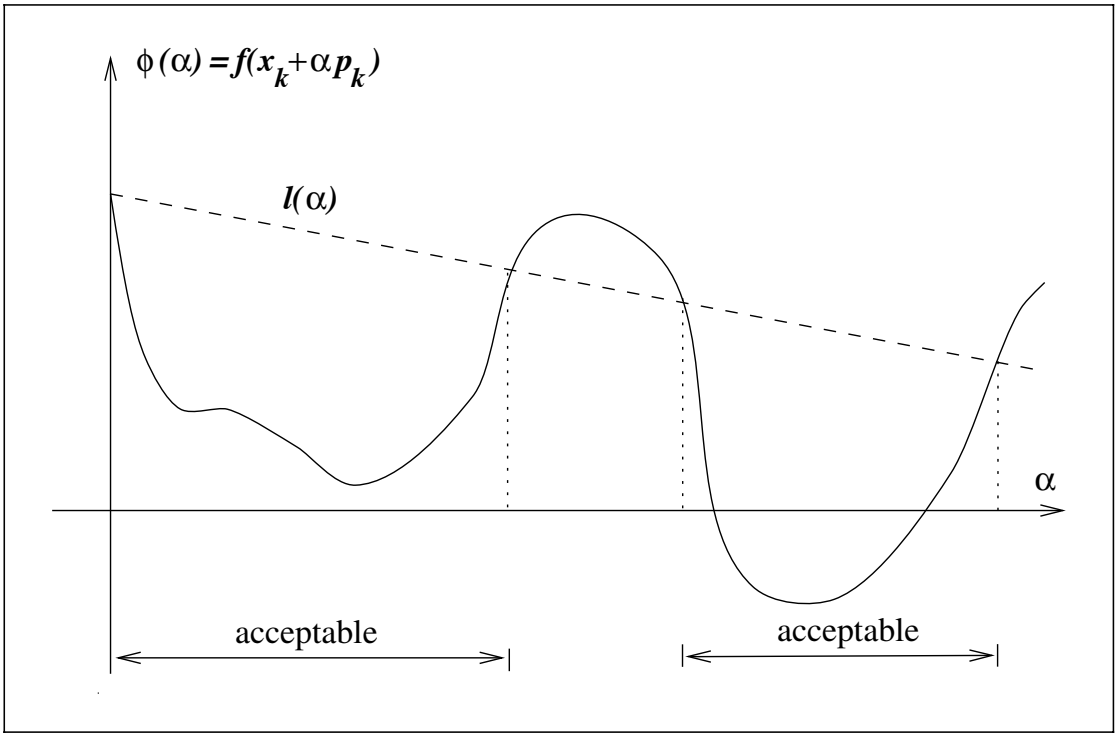


Figure 3.3 Sufficient decrease condition.

for some constant $c_1 \in (0, 1)$. In other words, the reduction in f should be proportional to both the step length α_k and the directional derivative $\nabla f_k^T p_k$. Inequality (3.4) is sometimes called the *Armijo condition.*

The sufficient decrease condition is illustrated in Figure 3.3. The right-hand-side of (3.4), which is a linear function, can be denoted by $l(\alpha)$. The function $l(\cdot)$ has negative slope $c_1 \nabla f_k^T p_k$, but because $c_1 \in (0, 1)$, it lies above the graph of ϕ for small positive values of α. The sufficient decrease condition states that α is acceptable only if $\phi(\alpha) \leq l(\alpha)$. The intervals on which this condition is satisfied are shown in Figure 3.3. In practice, c_1 is chosen to be quite small, say $c_1 = 10^{-4}$.

The sufficient decrease condition is not enough by itself to ensure that the algorithm makes reasonable progress, because as we see from Figure 3.3, it is satisfied for all sufficiently small values of α. To rule out unacceptably short steps we introduce a second requirement, called the *curvature condition*, which requires α_k to satisfy

$$\nabla f(x_k + \alpha_k p_k)^T p_k \geq c_2 \nabla f_k^T p_k, \tag{3.5}$$

for some constant $c_2 \in (c_1, 1)$, where c_1 is the constant from (3.4). Note that the left-hand-side is simply the derivative $\phi'(\alpha_k)$, so the curvature condition ensures that the slope of $\phi(\alpha_k)$ is greater than c_2 times the gradient $\phi'(0)$. This makes sense because if the slope $\phi'(\alpha)$ is strongly negative, we have an indication that we can reduce f significantly by moving further along the chosen direction. On the other hand, if the slope is only slightly negative or even positive, it is a sign that we cannot expect much more decrease in f in this direction,

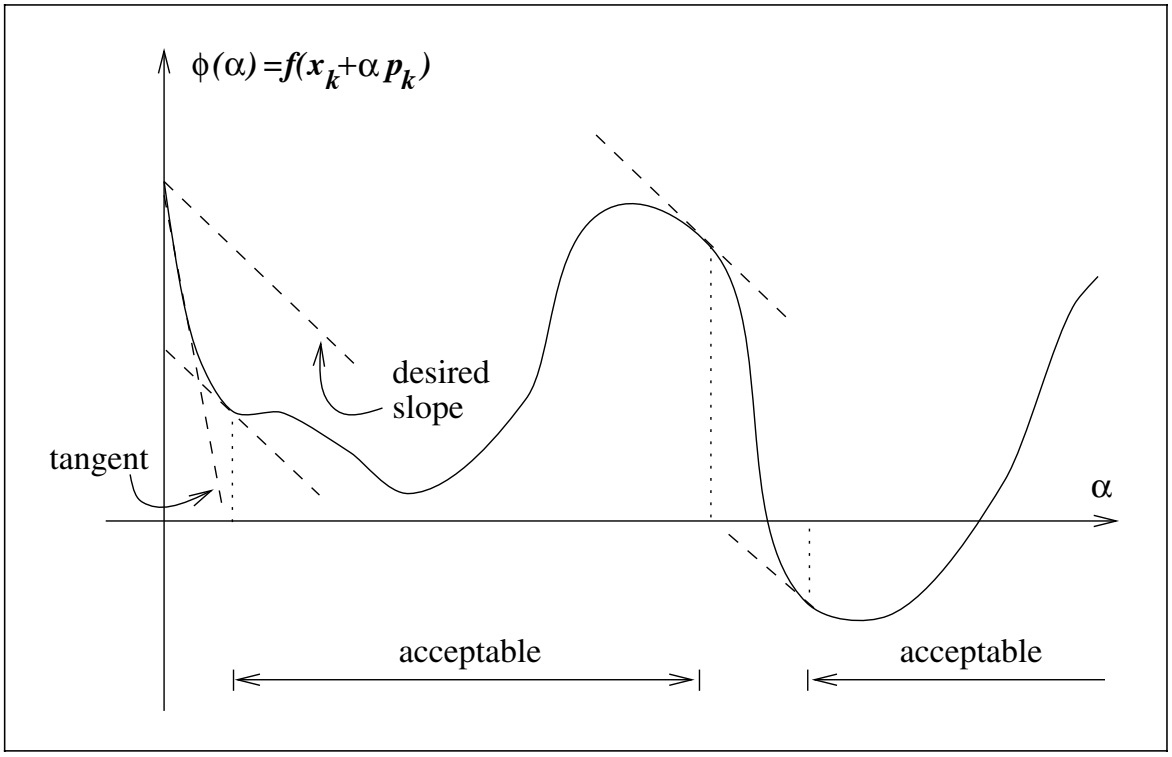


Figure 3.4 The curvature condition.

so it might make sense to terminate the line search. The curvature condition is illustrated in Figure 3.4. Typical values of c_2 are 0.9 when the search direction p_k is chosen by a Newton or quasi-Newton method, and 0.1 when p_k is obtained from a nonlinear conjugate gradient method.

The sufficient decrease and curvature conditions are known collectively as the *Wolfe conditions*. We illustrate them in Figure 3.5 and restate them here for future reference:

$$f(x_k + \alpha_k p_k) \leq f(x_k) + c_1 \alpha_k \nabla f_k^T p_k, \tag{3.6a}$$

$$\nabla f(x_k + \alpha_k p_k)^T p_k \geq c_2 \nabla f_k^T p_k, \tag{3.6b}$$

with $0 < c_1 < c_2 < 1$.

A step length may satisfy the Wolfe conditions without being particularly close to a minimizer of ϕ, as we show in Figure 3.5. We can, however, modify the curvature condition to force α_k to lie in at least a broad neighborhood of a local minimizer or stationary point of ϕ. The *strong Wolfe conditions* require α_k to satisfy

$$f(x_k + \alpha_k p_k) \leq f(x_k) + c_1 \alpha_k \nabla f_k^T p_k, \tag{3.7a}$$

$$|\nabla f(x_k + \alpha_k p_k)^T p_k| \leq c_2 |\nabla f_k^T p_k|, \tag{3.7b}$$

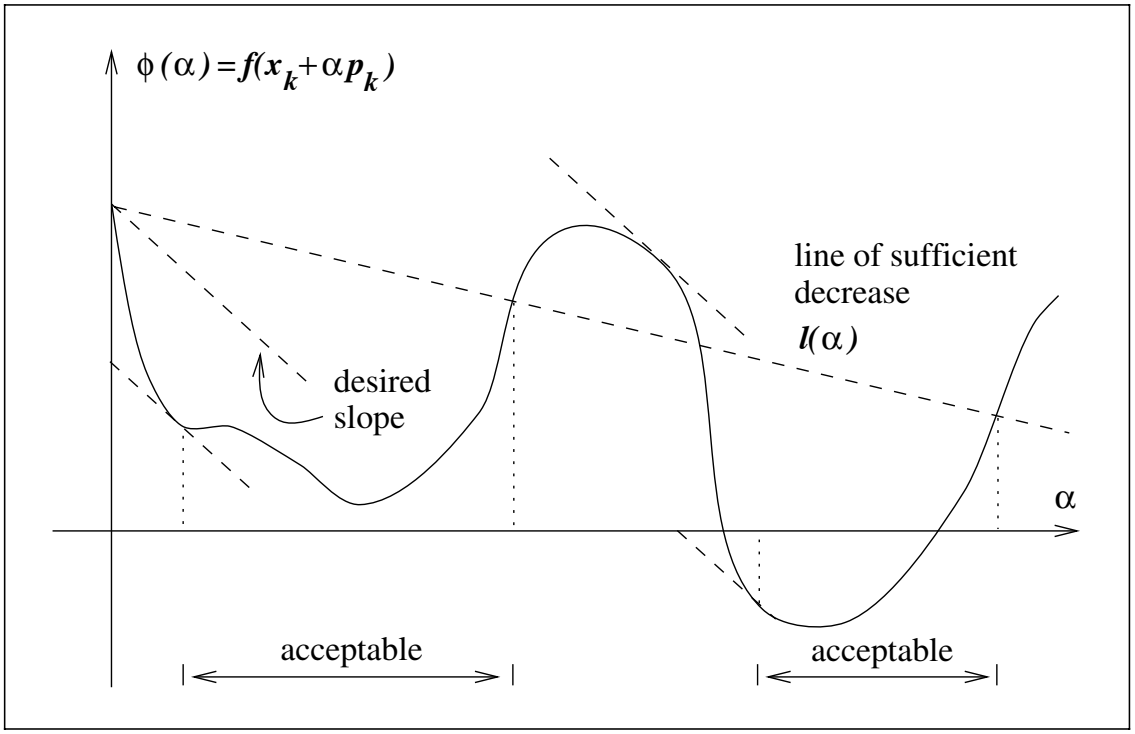


Figure 3.5 Step lengths satisfying the Wolfe conditions.

with $0 < c_1 < c_2 < 1$. The only difference with the Wolfe conditions is that we no longer allow the derivative $\phi'(\alpha_k)$ to be too positive. Hence, we exclude points that are far from stationary points of ϕ.

It is not difficult to prove that there exist step lengths that satisfy the Wolfe conditions for every function f that is smooth and bounded below.

Lemma 3.1.

Suppose that $f : \mathbb{R}^n \to \mathbb{R}$ is continuously differentiable. Let p_k be a descent direction at x_k, and assume that f is bounded below along the ray $\{x_k + \alpha p_k | \alpha > 0\}$. Then if $0 < c_1 < c_2 < 1$, there exist intervals of step lengths satisfying the Wolfe conditions (3.6) and the strong Wolfe conditions (3.7).

PROOF. Since $\phi(\alpha) = f(x_k + \alpha p_k)$ is bounded below for all $\alpha > 0$ and since $0 < c_1 < 1$, the line $l(\alpha) = f(x_k) + \alpha c_1 \nabla f_k^T p_k$ must intersect the graph of ϕ at least once. Let $\alpha' > 0$ be the smallest intersecting value of α, that is,

$$f(x_k + \alpha' p_k) = f(x_k) + \alpha' c_1 \nabla f_k^T p_k. \tag{3.8}$$

The sufficient decrease condition (3.6a) clearly holds for all step lengths less than α'.

By the mean value theorem, there exists $\alpha'' \in (0, \alpha')$ such that

$$f(x_k + \alpha' p_k) - f(x_k) = \alpha' \nabla f(x_k + \alpha'' p_k)^T p_k. \tag{3.9}$$

By combining (3.8) and (3.9), we obtain

$$\nabla f(x_k + \alpha'' p_k)^T p_k = c_1 \nabla f_k^T p_k > c_2 \nabla f_k^T p_k, \tag{3.10}$$

since $c_1 < c_2$ and $\nabla f_k^T p_k < 0$. Therefore, α'' satisfies the Wolfe conditions (3.6), and the inequalities hold strictly in both (3.6a) and (3.6b). Hence, by our smoothness assumption on f, there is an interval around α'' for which the Wolfe conditions hold. Moreover, since the term in the left-hand side of (3.10) is negative, the strong Wolfe conditions (3.7) hold in the same interval. □

The Wolfe conditions are scale-invariant in a broad sense: Multiplying the objective function by a constant or making an affine change of variables does not alter them. They can be used in most line search methods, and are particularly important in the implementation of quasi-Newton methods, as we see in Chapter 8.

THE GOLDSTEIN CONDITIONS

Like the Wolfe conditions, the *Goldstein conditions* also ensure that the step length α achieves sufficient decrease while preventing α from being too small. The Goldstein conditions can also be stated as a pair of inequalities, in the following way:

$$f(x_k) + (1-c)\alpha_k \nabla f_k^T p_k \leq f(x_k + \alpha_k p_k) \leq f(x_k) + c\alpha_k \nabla f_k^T p_k, \tag{3.11}$$

with $0 < c < \frac{1}{2}$. The second inequality is the sufficient decrease condition (3.4), whereas the first inequality is introduced to control the step length from below; see Figure 3.6

A disadvantage of the Goldstein conditions vis-à-vis the Wolfe conditions is that the first inequality in (3.11) may exclude all minimizers of ϕ. However, the Goldstein and Wolfe conditions have much in common, and their convergence theories are quite similar. The Goldstein conditions are often used in Newton-type methods but are not well suited for quasi-Newton methods that maintain a positive definite Hessian approximation.

SUFFICIENT DECREASE AND BACKTRACKING

We have mentioned that the sufficient decrease condition (3.6a) alone is not sufficient to ensure that the algorithm makes reasonable progress along the given search direction. However, if the line search algorithm chooses its candidate step lengths appropriately, by using a so-called *backtracking* approach, we can dispense with the extra condition (3.6b) and use just the sufficient decrease condition to terminate the line search procedure. In its most basic form, backtracking proceeds as follows.

Procedure 3.1 (Backtracking Line Search).
Choose $\bar{\alpha} > 0$, $\rho, c \in (0, 1)$; set $\alpha \leftarrow \bar{\alpha}$;
repeat until $f(x_k + \alpha p_k) \leq f(x_k) + c\alpha \nabla f_k^T p_k$

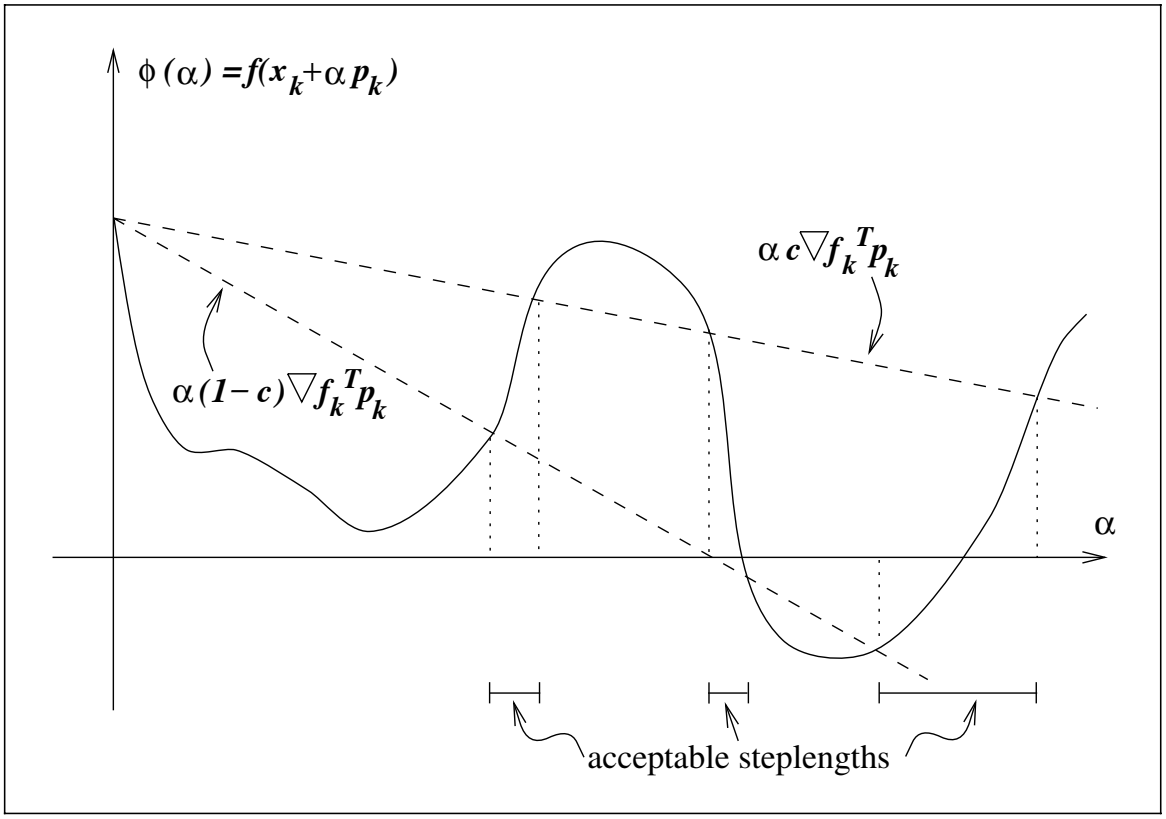


Figure 3.6 The Goldstein conditions.

$\alpha \leftarrow \rho\alpha$;
end (repeat)
Terminate with $\alpha_k = \alpha$.

In this procedure, the initial step length $\bar{\alpha}$ is chosen to be 1 in Newton and quasi-Newton methods, but can have different values in other algorithms such as steepest descent or conjugate gradient. An acceptable step length will be found after a finite number of trials because α_k will eventually become small enough that the sufficient decrease condition holds (see Figure 3.3). In practice, the contraction factor ρ is often allowed to vary at each iteration of the line search. For example, it can be chosen by safeguarded interpolation, as we describe later. We need ensure only that at each iteration we have $\rho \in [\rho_{\text{lo}}, \rho_{\text{hi}}]$, for some fixed constants $0 < \rho_{\text{lo}} < \rho_{\text{hi}} < 1$.

The backtracking approach ensures either that the selected step length α_k is some fixed value (the initial choice $\bar{\alpha}$), or else that it is short enough to satisfy the sufficient decrease condition but not *too* short. The latter claim holds because the accepted value α_k is within striking distance of the previous trial value, α_k/ρ, which was rejected for violating the sufficient decrease condition, that is, for being too long.

This simple and popular strategy for terminating a line search is well suited for Newton methods (see Chapter 6) but is less appropriate for quasi-Newton and conjugate gradient methods.

3.2 CONVERGENCE OF LINE SEARCH METHODS

To obtain global convergence, we must not only have well-chosen step lengths but also well-chosen search directions p_k. We discuss requirements on the search direction in this section, focusing on one key property: the angle θ_k between p_k and the steepest descent direction $-\nabla f_k$, defined by

$$\cos\theta_k = \frac{-\nabla f_k^T p_k}{\|\nabla f_k\| \, \|p_k\|}. \tag{3.12}$$

The following theorem, due to Zoutendijk, has far-reaching consequences. It shows, for example, that the steepest descent method is globally convergent. For other algorithms it describes how far p_k can deviate from the steepest descent direction and still give rise to a globally convergent iteration. Various line search termination conditions can be used to establish this result, but for concreteness we will consider only the Wolfe conditions (3.6). Though Zoutendijk's result appears, at first, to be technical and obscure, its power will soon become evident.

Theorem 3.2.

Consider any iteration of the form (3.1), where p_k is a descent direction and α_k satisfies the Wolfe conditions (3.6). Suppose that f is bounded below in $\mathbb{R}^n$ and that f is continuously differentiable in an open set $\mathcal{N}$ containing the level set $\mathcal{L} \stackrel{\text{def}}{=} \{x : f(x) \le f(x_0)\}$, where x_0 is the starting point of the iteration. Assume also that the gradient ∇f is Lipschitz continuous on $\mathcal{N}$, that is, there exists a constant $L > 0$ such that

$$\|\nabla f(x) - \nabla f(\tilde{x})\| \le L\|x - \tilde{x}\|, \quad \text{for all } x, \tilde{x} \in \mathcal{N}. \tag{3.13}$$

Then

$$\sum_{k\ge 0} \cos^2\theta_k \, \|\nabla f_k\|^2 < \infty. \tag{3.14}$$

PROOF. From (3.6b) and (3.1) we have that

$$(\nabla f_{k+1} - \nabla f_k)^T p_k \ge (c_2 - 1)\nabla f_k^T p_k,$$

while the Lipschitz condition (3.13) implies that

$$(\nabla f_{k+1} - \nabla f_k)^T p_k \leq \alpha_k L \|p_k\|^2.$$

By combining these two relations, we obtain

$$\alpha_k \geq \frac{c_2 - 1}{L} \frac{\nabla f_k^T p_k}{\|p_k\|^2}.$$

By substituting this inequality into the first Wolfe condition (3.6a), we obtain

$$f_{k+1} \leq f_k - c_1 \frac{1 - c_2}{L} \frac{(\nabla f_k^T p_k)^2}{\|p_k\|^2}.$$

From the definition (3.12), we can write this relation as

$$f_{k+1} \leq f_k - c \cos^2 \theta_k \|\nabla f_k\|^2,$$

where $c = c_1(1 - c_2)/L$. By summing this expression over all indices less than or equal to k, we obtain

$$f_{k+1} \leq f_0 - c \sum_{j=0}^{k} \cos^2 \theta_j \|\nabla f_j\|^2. \tag{3.15}$$

Since f is bounded below, we have that $f_0 - f_{k+1}$ is less than some positive constant, for all k. Hence by taking limits in (3.15), we obtain

$$\sum_{k=0}^{\infty} \cos^2 \theta_k \|\nabla f_k\|^2 < \infty,$$

which concludes the proof. □

Similar results to this theorem hold when the Goldstein conditions (3.11) or strong Wolfe conditions (3.7) are used in place of the Wolfe conditions.

Note that the assumptions of Theorem 3.2 are not too restrictive. If the function f were not bounded below, the optimization problem would not be well-defined. The smoothness assumption—Lipschitz continuity of the gradient—is implied by many of the smoothness conditions that are used in local convergence theorems (see Chapters 6 and 8) and are often satisfied in practice.

Inequality (3.14), which we call the *Zoutendijk condition*, implies that

$$\cos^2 \theta_k \|\nabla f_k\|^2 \to 0. \tag{3.16}$$

This limit can be used in turn to derive global convergence results for line search algorithms.

If our method for choosing the search direction p_k in the iteration (3.1) ensures that the angle θ_k defined by (3.12) is bounded away from 90°, there is a positive constant δ such that

$$\cos\theta_k \geq \delta > 0, \qquad \text{for all } k. \tag{3.17}$$

It follows immediately from (3.16) that

$$\lim_{k\to\infty} \|\nabla f_k\| = 0. \tag{3.18}$$

In other words, we can be sure that the gradient norms $\|\nabla f_k\|$ converge to zero, provided that the search directions are never too close to orthogonality with the gradient. In particular, the method of steepest descent (for which the search direction p_k makes an angle of *zero* degrees with the negative gradient) produces a gradient sequence that converges to zero, provided that it uses a line search satisfying the Wolfe or Goldstein conditions.

We use the term *globally convergent* to refer to algorithms for which the property (3.18) is satisfied, but note that this term is sometimes used in other contexts to mean different things. For line search methods of the general form (3.1), the limit (3.18) is the strongest global convergence result that can be obtained: We cannot guarantee that the method converges to a minimizer, but only that it is attracted by stationary points. Only by making additional requirements on the search direction p_k—by introducing negative curvature information from the Hessian $\nabla^2 f(x_k)$, for example—can we strengthen these results to include convergence to a local minimum. See the Notes and References at the end of this chapter for further discussion of this point.

Consider now the Newton-like method (3.1), (3.2) and assume that the matrices B_k are positive definite with a uniformly bounded condition number. That is, there is a constant M such that

$$\|B_k\| \, \|B_k^{-1}\| \leq M, \qquad \text{for all } k.$$

It is easy to show from the definition (3.12) that

$$\cos\theta_k \geq 1/M \tag{3.19}$$

(see Exercise 5). By combining this bound with (3.16) we find that

$$\lim_{k\to\infty} \|\nabla f_k\| = 0. \tag{3.20}$$

Therefore, we have shown that Newton and quasi-Newton methods are globally convergent if the matrices B_k have a bounded condition number and are positive definite (which is

needed to ensure that p_k is a descent direction), and if the step lengths satisfy the Wolfe conditions.

For some algorithms, such as conjugate gradient methods, we will not be able to prove the limit (3.18), but only the weaker result

$$\liminf_{k\to\infty} \|\nabla f_k\| = 0. \tag{3.21}$$

In other words, just a subsequence of the gradient norms $\|\nabla f_{k_j}\|$ converges to zero, rather than the whole sequence (see Appendix A). This result, too, can be proved by using Zoutendijk's condition (3.14), but instead of a constructive proof, we outline a proof by contradiction. Suppose that (3.21) does not hold, so that the gradients remain bounded away from zero, that is, there exists $\gamma > 0$ such that

$$\|\nabla f_k\| \geq \gamma, \quad \text{for all } k \text{ sufficiently large.} \tag{3.22}$$

Then from (3.16) we conclude that

$$\cos\theta_k \to 0, \tag{3.23}$$

that is, the entire sequence $\{\cos\theta_k\}$ converges to 0. To establish (3.21), therefore, it is enough to show that a subsequence $\{\cos\theta_{k_j}\}$ is bounded away from zero. We will use this strategy in Chapter 5 to study the convergence of nonlinear conjugate gradient methods.

By applying this proof technique, we can prove global convergence in the sense of (3.20) or (3.21) for a general class of algorithms. Consider *any* algorithm for which (i) every iteration produces a decrease in the objective function, and (ii) every mth iteration is a steepest descent step, with step length chosen to satisfy the Wolfe or Goldstein conditions. Then since $\cos\theta_k = 1$ for the steepest descent steps, the result (3.20) holds. Of course, we would design the algorithm so that it does something "better" than steepest descent at the other $m-1$ iterates; the occasional steepest descent steps may not make much progress, but they at least guarantee overall global convergence.

Note that throughout this section we have used only the fact that Zoutendijk's condition implies the limit (3.16). In later chapters we will make use of the bounded sum condition (3.14), which forces the sequence $\{\cos^2\theta_k\|\nabla f_k\|^2\}$ to converge to zero at a sufficiently rapid rate.

3.3 RATE OF CONVERGENCE

It would seem that designing optimization algorithms with good convergence properties is easy, since all we need to ensure is that the search direction p_k does not tend to become orthogonal to the gradient ∇f_k, or that steepest descent steps are taken regularly. We could

simply compute $\cos\theta_k$ at every iteration and turn p_k toward the steepest descent direction if $\cos\theta_k$ is smaller than some preselected constant $\delta > 0$. Angle tests of this type ensure global convergence, but they are undesirable for two reasons. First, they may impede a fast rate of convergence, because for problems with an ill-conditioned Hessian, it may be necessary to produce search directions that are almost orthogonal to the gradient, and an inappropriate choice of the parameter δ may prevent this. Second, angle tests destroy the invariance properties of quasi-Newton methods.

Algorithmic strategies that achieve rapid convergence can sometimes conflict with the requirements of global convergence, and vice versa. For example, the steepest descent method is the quintessential globally convergent algorithm, but it is quite slow in practice, as we shall see below. On the other hand, the pure Newton iteration converges rapidly when started close enough to a solution, but its steps may not even be descent directions away from the solution. The challenge is to design algorithms that incorporate both properties: good global convergence guarantees and a rapid rate of convergence.

We begin our study of convergence rates of line search methods by considering the most basic approach of all: the steepest descent method.

CONVERGENCE RATE OF STEEPEST DESCENT

We can learn much about the steepest descent method by considering the ideal case, in which the objective function is quadratic and the line searches are exact. Let us suppose that

$$f(x) = \tfrac{1}{2}x^T Qx - b^T x, \tag{3.24}$$

where Q is symmetric and positive definite. The gradient is given by $\nabla f(x) = Qx - b$, and the minimizer x^* is the unique solution of the linear system $Qx = b$.

Let us compute the step length α_k that minimizes $f(x_k - \alpha\nabla f_k)$. By differentiating

$$f(x_k - \alpha g_k) = \frac{1}{2}(x_k - \alpha g_k)^T Q(x_k - \alpha g_k) - b^T(x_k - \alpha g_k)$$

with respect to α, we obtain

$$\alpha_k = \frac{\nabla f_k^T \nabla f_k}{\nabla f_k^T Q \nabla f_k}. \tag{3.25}$$

If we use this exact minimizer α_k, the steepest descent iteration for (3.24) is given by

$$x_{k+1} = x_k - \left(\frac{\nabla f_k^T \nabla f_k}{\nabla f_k^T Q \nabla f_k}\right)\nabla f_k. \tag{3.26}$$

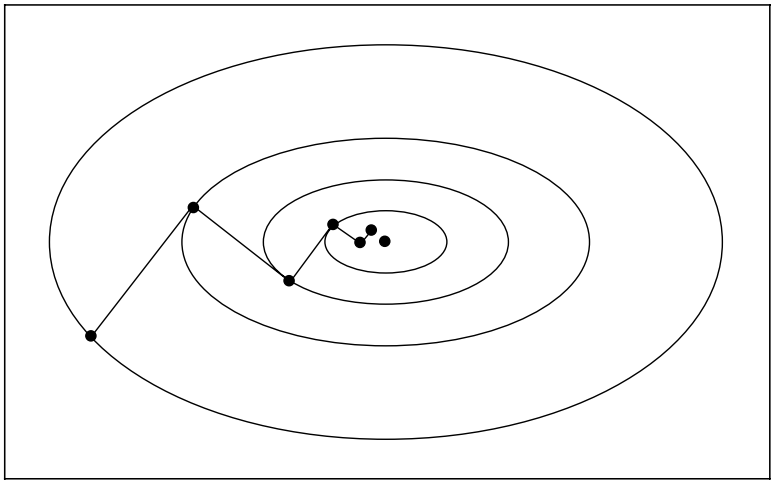


Figure 3.7 Steepest descent steps.

Since $\nabla f_k = Qx_k - b$, this equation yields a closed-form expression for x_{k+1} in terms of x_k. In Figure 3.7 we plot a typical sequence of iterates generated by the steepest descent method on a two-dimensional quadratic objective function. The contours of f are ellipsoids whose axes lie along the orthogonal eigenvectors of Q. Note that the iterates zigzag toward the solution.

To quantify the rate of convergence we introduce the weighted norm $\|x\|_Q^2 = x^T Qx$. By using the relation $Qx^* = b$, we can show easily that

$$\tfrac{1}{2}\|x - x^*\|_Q^2 = f(x) - f(x^*), \tag{3.27}$$

so that this norm measures the difference between the current objective value and the optimal value. By using the equality (3.26) and noting that $\nabla f_k = Q(x_k - x^*)$, we can derive the equality

$$\|x_{k+1} - x^*\|_Q^2 = \left\{1 - \frac{\left(\nabla f_k^T \nabla f_k\right)^2}{\left(\nabla f_k^T Q \nabla f_k\right)\left(\nabla f_k^T Q^{-1} \nabla f_k\right)}\right\} \|x_k - x^*\|_Q^2 \tag{3.28}$$

(see Exercise 7). This expression describes the exact decrease in f at each iteration, but since the term inside the brackets is difficult to interpret, it is more useful to bound it in terms of the condition number of the problem.

Theorem 3.3.

When the steepest descent method with exact line searches (3.26) is applied to the strongly convex quadratic function (3.24), the error norm (3.27) satisfies

$$\|x_{k+1} - x^*\|_Q^2 \leq \left(\frac{\lambda_n - \lambda_1}{\lambda_n + \lambda_1}\right)^2 \|x_k - x^*\|_Q^2, \tag{3.29}$$

where $0 < \lambda_1 \leq \cdots \leq \lambda_n$ *are the eigenvalues of* Q.

The proof of this result is given by Luenberger [152]. The inequalities (3.29) and (3.27) show that the function values f_k converge to the minimum f_* at a linear rate. As a special case of this result, we see that convergence is achieved in one iteration if all the eigenvalues are equal. In this case, Q is a multiple of the identity matrix, so the contours in Figure 3.7 are circles and the steepest descent direction always points at the solution. In general, as the condition number $\kappa(Q) = \lambda_n/\lambda_1$ increases, the contours of the quadratic become more elongated, the zigzagging in Figure 3.7 becomes more pronounced, and (3.29) implies that the convergence degrades. Even though (3.29) is a worst-case bound, it gives an accurate indication of the behavior of the algorithm when $n > 2$.

The rate-of-convergence behavior of the steepest descent method is essentially the same on general nonlinear objective functions. In the following result we assume that the step length is the global minimizer along the search direction.

Theorem 3.4.

Suppose that $f : \mathbb{R}^n \to \mathbb{R}$ *is twice continuously differentiable, and that the iterates generated by the steepest descent method with exact line searches converge to a point* x^* *where the Hessian matrix* $\nabla^2 f(x^*)$ *is positive definite. Then*

$$f(x_{k+1}) - f(x^*) \leq \left(\frac{\lambda_n - \lambda_1}{\lambda_n + \lambda_1}\right)^2 [f(x_k) - f(x^*)],$$

where $\lambda_1 \leq \cdots \leq \lambda_n$ *are the eigenvalues of* $\nabla^2 f(x^*)$.

In general, we cannot expect the rate of convergence to improve if an inexact line search is used. Therefore, Theorem 3.4 shows that the steepest descent method can have an unacceptably slow rate of convergence, even when the Hessian is reasonably well conditioned. For example, if $\kappa(Q) = 800$, $f(x_1) = 1$ and $f(x^*) = 0$, Theorem 3.4 suggests that the function value will still be about 0.08 after one thousand iterations of the steepest descent method.

QUASI-NEWTON METHODS

Let us now suppose that the search direction has the form

$$p_k = -B_k^{-1} \nabla f_k, \tag{3.30}$$

where the symmetric and positive definite matrix B_k is updated at every iteration by a quasi-Newton updating formula. We already encountered one quasi-Newton formula, the BFGS formula, in Chapter 2; others will be discussed in Chapter 8. We assume here that the step length α_k will be computed by an inexact line search that satisfies the Wolfe or strong Wolfe

conditions, with one important proviso: The line search algorithm will always try the step length $\alpha = 1$ first, and will accept this value if it satisfies the Wolfe conditions. (We could enforce this condition by setting $\bar{\alpha} = 1$ in Procedure 3.1, for example.) This implementation detail turns out to be crucial in obtaining a fast rate of convergence.

The following result, due to Dennis and Moré, shows that if the search direction of a quasi-Newton method approximates the Newton direction well enough, then the unit step length will satisfy the Wolfe conditions as the iterates converge to the solution. It also specifies a condition that the search direction must satisfy in order to give rise to a superlinearly convergent iteration. To bring out the full generality of this result, we state it first in terms of a general descent iteration, and then examine its consequences for quasi-Newton and Newton methods.

Theorem 3.5.

Suppose that $f : \mathbb{R}^n \to \mathbb{R}$ is three times continuously differentiable. Consider the iteration $x_{k+1} = x_k + \alpha_k p_k$, where p_k is a descent direction and α_k satisfies the Wolfe conditions (3.6) with $c_1 \leq \frac{1}{2}$. If the sequence $\{x_k\}$ converges to a point x^* such that $\nabla f(x^*) = 0$ and $\nabla^2 f(x^*)$ is positive definite, and if the search direction satisfies

$$\lim_{k\to\infty} \frac{\|\nabla f_k + \nabla^2 f_k p_k\|}{\|p_k\|} = 0, \tag{3.31}$$

then

(i) *the step length $\alpha_k = 1$ is admissible for all k greater than a certain index k_0; and*

(ii) *if $\alpha_k = 1$ for all $k > k_0$, $\{x_k\}$ converges to x^* superlinearly.*

It is easy to see that if $c_1 > \frac{1}{2}$, then the line search would exclude the minimizer of a quadratic, and unit step lengths may not be admissible.

If p_k is a quasi-Newton search direction of the form (3.30), then (3.31) is equivalent to

$$\lim_{k\to\infty} \frac{\|(B_k - \nabla^2 f(x^*))p_k\|}{\|p_k\|} = 0. \tag{3.32}$$

Hence, we have the surprising (and delightful) result that a superlinear convergence rate can be attained even if the sequence of quasi-Newton matrices B_k does not converge to $\nabla^2 f(x^*)$; it suffices that the B_k become increasingly accurate approximations to $\nabla^2 f(x^*)$ *along the search directions* p_k.

An important remark is that condition (3.32) is both necessary and sufficient for the superlinear convergence of quasi-Newton methods.

Theorem 3.6.

Suppose that $f : \mathbb{R}^n \to \mathbb{R}$ is three times continuously differentiable. Consider the iteration $x_{k+1} = x_k + p_k$ (that is, the step length α_k is uniformly 1) and that p_k is given by (3.30). Let us also assume that $\{x_k\}$ converges to a point x^* such that $\nabla f(x^*) = 0$ and $\nabla^2 f(x^*)$ is positive definite. Then $\{x_k\}$ converges superlinearly if and only if (3.32) holds.

PROOF. We first show that (3.32) is equivalent to

$$p_k - p_k^{\mathrm{N}} = o(\|p_k\|), \tag{3.33}$$

where $p_k^{\mathrm{N}} = -\nabla^2 f_k^{-1} \nabla f_k$ is the Newton step. Assuming that (3.32) holds, we have that

$$\begin{aligned} p_k - p_k^{\mathrm{N}} &= \nabla^2 f_k^{-1}(\nabla^2 f_k p_k + \nabla f_k) \\ &= \nabla^2 f_k^{-1}(\nabla^2 f_k - B_k) p_k \\ &= O(\|(\nabla^2 f_k - B_k) p_k\|) \\ &= o(\|p_k\|), \end{aligned}$$

where we have used the fact that $\|\nabla^2 f_k^{-1}\|$ is bounded above for x_k sufficiently close to x^*, since the limiting Hessian $\nabla^2 f(x^*)$ is positive definite. The converse follows readily if we multiply both sides of (3.33) by $\nabla^2 f_k$ and recall (3.30).

For the remainder of the proof we need to look ahead to the proof of quadratic convergence of Newton's method and, in particular, the estimate (3.37). By using this inequality together with (3.33), we obtain that

$$\begin{aligned} \|x_k + p_k - x^*\| &\le \|x_k + p_k^{\mathrm{N}} - x^*\| + \|p_k - p_k^{\mathrm{N}}\| \\ &= O(\|x_k - x^*\|^2) + o(\|p_k\|). \end{aligned}$$

A simple manipulation of this inequality reveals that $\|p_k\| = O(\|x_k - x^*\|)$, so we obtain

$$\|x_k + p_k - x^*\| \le o(\|x_k - x^*\|),$$

giving the superlinear convergence result. □

We will see in Chapter 8 that quasi-Newton methods normally satisfy condition (3.32) and are superlinearly convergent.

NEWTON'S METHOD

Let us now consider the Newton iteration where the search direction is given by

$$p_k^{\mathrm{N}} = -\nabla^2 f_k^{-1} \nabla f_k. \tag{3.34}$$

Since the Hessian matrix $\nabla^2 f_k$ may not always be positive definite, p_k^{N} may not always be a descent direction, and many of the ideas discussed so far in this chapter no longer apply. In Chapter 6 we will describe two approaches for obtaining a globally convergent iteration based on the Newton step: a line search approach, in which the Hessian $\nabla^2 f_k$ is modified, if necessary, to make it positive definite and thereby yield descent, and a trust region approach, in which $\nabla^2 f_k$ is used to form a quadratic model that is minimized in a ball.

Here we discuss just the local rate-of-convergence properties of Newton's method. We know that for all x in the vicinity of a solution point x^* such that $\nabla^2 f(x^*)$ is positive definite, the Hessian $\nabla^2 f(x)$ will also be positive definite. Newton's method will be well-defined in this region and will converge quadratically, provided that the step lengths α_k are eventually always 1.

Theorem 3.7.

Suppose that f is twice differentiable and that the Hessian $\nabla^2 f(x)$ is Lipschitz continuous (see (A.8)) in a neighborhood of a solution x^* at which the sufficient conditions (Theorem 2.4) are satisfied. Consider the iteration $x_{k+1} = x_k + p_k$, where p_k is given by (3.34). Then

1. *if the starting point x_0 is sufficiently close to x^*, the sequence of iterates converges to x^*;*
2. *the rate of convergence of $\{x_k\}$ is quadratic; and*
3. *the sequence of gradient norms $\{\|\nabla f_k\|\}$ converges quadratically to zero.*

PROOF. From the definition of the Newton step and the optimality condition $\nabla f_* = 0$ we have that

$$\begin{aligned} x_k + p_k^{\mathrm{N}} - x^* &= x_k - x^* - \nabla^2 f_k^{-1} \nabla f_k \\ &= \nabla^2 f_k^{-1} \left[\nabla^2 f_k (x_k - x^*) - (\nabla f_k - \nabla f_*) \right]. \end{aligned} \tag{3.35}$$

Since

$$\nabla f_k - \nabla f_* = \int_0^1 \nabla^2 f(x_k + t(x^* - x_k))(x_k - x^*)\, dt,$$

we have

$$\begin{aligned} &\left\| \nabla^2 f(x_k)(x_k - x^*) - (\nabla f_k - \nabla f(x^*)) \right\| \\ &\quad = \left\| \int_0^1 \left[\nabla^2 f(x_k) - \nabla^2 f(x_k + t(x^* - x_k)) \right] (x_k - x^*)\, dt \right\| \\ &\quad \le \int_0^1 \left\| \nabla^2 f(x_k) - \nabla^2 f(x_k + t(x^* - x_k)) \right\| \|x_k - x^*\|\, dt \\ &\quad \le \|x_k - x^*\|^2 \int_0^1 Lt\, dt = \tfrac{1}{2} L \|x_k - x^*\|^2, \end{aligned} \tag{3.36}$$

where L is the Lipschitz constant for $\nabla^2 f(x)$ for x near x^*. Since $\nabla^2 f(x^*)$ is nonsingular, and since $\nabla^2 f_k \to \nabla^2 f(x^*)$, we have that $\|\nabla^2 f_k^{-1}\| \leq 2\|\nabla^2 f(x^*)^{-1}\|$ for all k sufficiently large. By substituting in (3.35) and (3.36), we obtain

$$\|x_k + p_k^{\mathrm{N}} - x^*\| \leq L\|\nabla^2 f(x^*)^{-1}\|\|x_k - x^*\|^2 = \tilde{L}\|x_k - x^*\|^2, \tag{3.37}$$

where $\tilde{L} = L\|\nabla^2 f(x^*)^{-1}\|$. Using this inequality inductively we deduce that if the starting point is sufficiently near x^*, then the sequence converges to x^*, and the rate of convergence is quadratic.

By using the relations $x_{k+1} - x_k = p_k^{\mathrm{N}}$ and $\nabla f_k + \nabla^2 f_k p_k^{\mathrm{N}} = 0$, we obtain that

$$\begin{aligned}
\|\nabla f(x_{k+1})\| &= \|\nabla f(x_{k+1}) - \nabla f_k - \nabla^2 f(x_k) p_k^{\mathrm{N}}\| \\
&= \left\| \int_0^1 \nabla^2 f(x_k + t\, p_k^{\mathrm{N}})(x_{k+1} - x_k)\, dt - \nabla^2 f(x_k) p_k^{\mathrm{N}} \right\| \\
&\leq \int_0^1 \left\| \nabla^2 f(x_k + t\, p_k^{\mathrm{N}}) - \nabla^2 f(x_k) \right\| \|p_k^{\mathrm{N}}\|\, dt \\
&\leq \tfrac{1}{2} L \|p_k^{\mathrm{N}}\|^2 \\
&\leq \tfrac{1}{2} L \|\nabla^2 f(x_k)^{-1}\|^2 \|\nabla f_k\|^2 \\
&\leq 2L \|\nabla^2 f(x^*)^{-1}\|^2 \|\nabla f_k\|^2,
\end{aligned}$$

proving that the gradient norms converge to zero quadratically. □

When the search direction is given by Newton's method, the limit (3.31) is satisfied (the ratio is zero for all k!), and Theorem 3.5 shows that the Wolfe conditions will accept the step length α_k for all large k. The same is true of the Goldstein conditions. Thus implementations of Newton's method using these conditions, and in which the line search always tries the unit step length first, will set $\alpha_k = 1$ for all large k and attain a local quadratic rate of convergence.

COORDINATE DESCENT METHODS

An approach that is frequently used in practice is to cycle through the n coordinate directions $e_1, e_2, \ldots, e_n$, using each in turn as a search direction. At the first iteration, we fix all except the first variable, and find a new value of this variable that minimizes (or at least reduces) the objective function. On the next iteration, we repeat the process with the *second* variable, and so on. After n iterations, we return to the first variable and repeat the cycle. The method is referred to as the *method of alternating variables* or the *coordinate descent method.* Though simple and somewhat intuitive, it can be quite inefficient in practice, as we illustrate in Figure 3.8 for a quadratic function in two variables. Note that after a few iterations, neither the vertical nor the horizontal move makes much progress toward the solution.

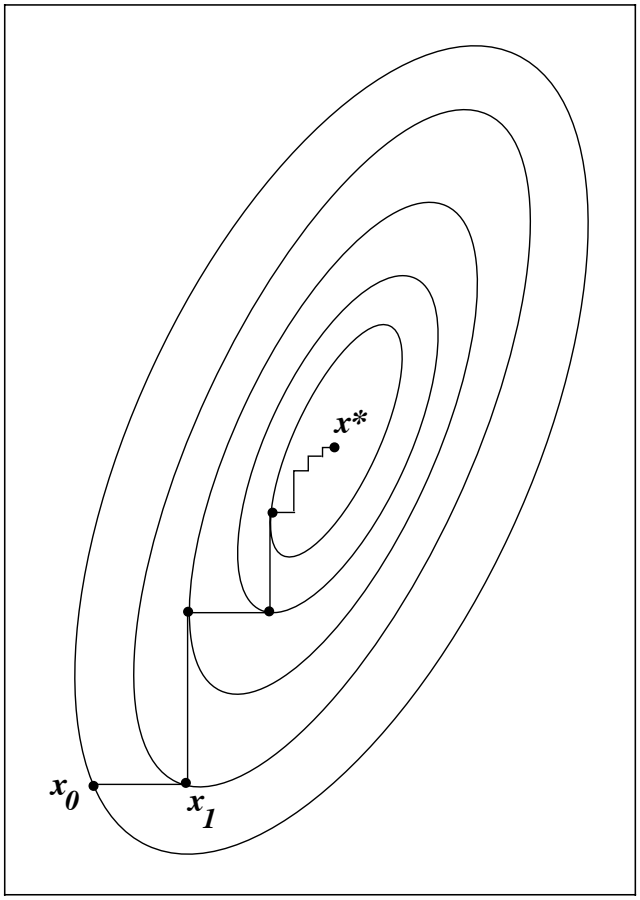


Figure 3.8
Coordinate descent.

In fact, the coordinate descent method with exact line searches can iterate infinitely without ever approaching a point where the gradient of the objective function vanishes. (By contrast, the steepest descent method produces a sequence for which $\|\nabla f_k\| \to 0$, as we showed earlier.) This observation can be generalized to show that a cyclic search along *any* set of linearly independent directions does not guarantee global convergence (Powell [198]). The difficulty that arises is that the gradient ∇f_k may become more and more perpendicular to the coordinate search direction, so that $\cos \theta_k$ approaches zero sufficiently rapidly that the Zoutendijk condition (3.14) is satisfied even when ∇f_k does not approach zero.

If the coordinate descent method converges to a solution, then its rate of convergence is often much slower than that of the steepest descent method, and the difference between them increases with the number of variables. However, the method may still be useful because

it does not require calculation of the gradient ∇f_k, and

the speed of convergence can be quite acceptable if the variables are loosely coupled.

Many variants of the coordinate descent method have been proposed, some of which are globally convergent. One simple variant is a "back-and-forth" approach in which we

search along the sequence of directions

$$e_1, e_2, \ldots, e_{n-1}, e_n, e_{n-1}, \ldots, e_2, e_1, e_2, \ldots \quad \text{(repeats)}.$$

Another approach, suggested by Figure 3.8, is first to perform a sequence of coordinate descent steps and then search along the line joining the first and last points in the cycle. Several algorithms, such as that of Hooke and Jeeves, are based on these ideas; see [104, 83].

3.4 STEP-LENGTH SELECTION ALGORITHMS

We now consider techniques for finding a minimum of the one-dimensional function

$$\phi(\alpha) = f(x_k + \alpha p_k), \tag{3.38}$$

or for simply finding a step length α_k satisfying one of the termination conditions described in Section 3.1. We assume that p_k is a descent direction—that is, $\phi'(0) < 0$—so that our search can be confined to positive values of α.

If f is a convex quadratic, $f(x) = \frac{1}{2}x^T Qx + b^T x + c$, its one-dimensional minimizer along the ray $x_k + \alpha p_k$ can be computed analytically and is given by

$$\alpha_k = -\frac{\nabla f_k^T p_k}{p_k^T Q p_k}. \tag{3.39}$$

For general nonlinear functions, it is necessary to use an iterative procedure. Much attention must be given to this line search because it has a major impact on the robustness and efficiency of all nonlinear optimization methods.

Line search procedures can be classified according to the type of derivative information they use. Algorithms that use only function values can be inefficient, since to be theoretically sound, they need to continue iterating until the search for the minimizer is narrowed down to a small interval. In contrast, knowledge of gradient information allows us to determine whether a suitable step length has been located, as stipulated, for example, by the Wolfe conditions (3.6) or Goldstein conditions (3.11). Often, particularly when the iterates are close to the solution, the very first step satisfies these conditions, so the line search need not be invoked at all. In the rest of this section we will discuss only algorithms that make use of derivative information. More information on derivative-free procedures is given in the notes at the end of this chapter.

All line search procedures require an initial estimate α_0 and generate a sequence $\{\alpha_i\}$ that either terminates with a step length satisfying the conditions specified by the user (for example, the Wolfe conditions) or determines that such a step length does not exist. Typical procedures consist of two phases: a *bracketing phase* that finds an interval $[a, b]$ containing



acceptable step lengths, and a *selection phase* that zooms in to locate the final step length. The selection phase usually reduces the bracketing interval during its search for the desired step length and interpolates some of the function and derivative information gathered on earlier steps to guess the location of the minimizer. We will first discuss how to perform this interpolation.

In the following discussion we let α_k and α_{k-1} denote the step lengths used at iterations k and $k-1$ of the optimization algorithm, respectively. On the other hand, we denote the trial step lengths generated during the line search by α_i and α_{i-1} and also α_j. We use α_0 to denote the initial guess.

INTERPOLATION

We begin by describing a line search procedure based on interpolation of known function and derivative values of the function ϕ. This procedure can be viewed as an enhancement of Procedure 3.1. The aim is to find a value of α that satisfies the sufficient decrease condition (3.6a), without being "too small." Accordingly, the procedures here generate a decreasing sequence of values α_i such that each value α_i is not too much smaller than its predecessor α_{i-1}.

Note that we can write the sufficient decrease condition in the notation of (3.38) as

$$\phi(\alpha_k) \leq \phi(0) + c_1\alpha_k\phi'(0), \tag{3.40}$$

and that since the constant c_1 is usually chosen to be small in practice ($c_1 = 10^{-4}$, say), this condition asks for little more than descent in f. We design the procedure to be "efficient" in the sense that it computes the derivative $\nabla f(x)$ as few times as possible.

Suppose that the initial guess α_0 is given. If we have

$$\phi(\alpha_0) \leq \phi(0) + c_1\alpha_0\phi'(0),$$

this step length satisfies the condition, and we terminate the search. Otherwise, we know that the interval $[0, \alpha_0]$ contains acceptable step lengths (see Figure 3.3). We form a quadratic approximation $\phi_q(\alpha)$ to ϕ by interpolating the three pieces of information available—$\phi(0)$, $\phi'(0)$, and $\phi(\alpha_0)$—to obtain

$$\phi_q(\alpha) = \left(\frac{\phi(\alpha_0) - \phi(0) - \alpha_0\phi'(0)}{\alpha_0^2}\right)\alpha^2 + \phi'(0)\alpha + \phi(0). \tag{3.41}$$

(Note that this function is constructed so that it satisfies the interpolation conditions $\phi_q(0) = \phi(0)$, $\phi_q'(0) = \phi'(0)$, and $\phi_q(\alpha_0) = \phi(\alpha_0)$.) The new trial value α_1 is defined as the minimizer of this quadratic, that is, we obtain

$$\alpha_1 = -\frac{\phi'(0)\alpha_0^2}{2\left[\phi(\alpha_0) - \phi(0) - \phi'(0)\alpha_0\right]}. \tag{3.42}$$

If the sufficient decrease condition (3.40) is satisfied at α_1, we terminate the search. Otherwise, we construct a *cubic* function that interpolates the four pieces of information $\phi(0)$, $\phi'(0)$, $\phi(\alpha_0)$, and $\phi(\alpha_1)$, obtaining

$$\phi_c(\alpha) = a\alpha^3 + b\alpha^2 + \alpha\phi'(0) + \phi(0),$$

where

$$\begin{bmatrix} a \\ b \end{bmatrix} = \frac{1}{\alpha_0^2\alpha_1^2(\alpha_1 - \alpha_0)} \begin{bmatrix} \alpha_0^2 & -\alpha_1^2 \\ -\alpha_0^3 & \alpha_1^3 \end{bmatrix} \begin{bmatrix} \phi(\alpha_1) - \phi(0) - \phi'(0)\alpha_1 \\ \phi(\alpha_0) - \phi(0) - \phi'(0)\alpha_0 \end{bmatrix}.$$

By differentiating $\phi_c(x)$, we see that the minimizer α_2 of ϕ_c lies in the interval $[0, \alpha_1]$ and is given by

$$\alpha_2 = \frac{-b + \sqrt{b^2 - 3a\phi'(0)}}{3a}.$$

If necessary, this process is repeated, using a cubic interpolant of $\phi(0)$, $\phi'(0)$ and the two most recent values of ϕ, until an α that satisfies (3.40) is located. If any α_i is either too close to its predecessor α_{i-1} or else too much smaller than α_{i-1}, we reset $\alpha_i = \alpha_{i-1}/2$. This safeguard procedure ensures that we make reasonable progress on each iteration and that the final α is not too small.

The strategy just described assumes that derivative values are significantly more expensive to compute than function values. It is often possible, however, to compute the directional derivative simultaneously with the function, at little additional cost; see Chapter 7. Accordingly, we can design an alternative strategy based on cubic interpolation of the values of ϕ and ϕ' at the two most recent values of α.

Cubic interpolation provides a good model for functions with significant changes of curvature. Suppose we have an interval $[a, b]$ known to contain desirable step lengths, and two previous step length estimates α_{i-1} and α_i in this interval. We use a cubic function to interpolate $\phi(\alpha_{i-1})$, $\phi'(\alpha_{i-1})$, $\phi(\alpha_i)$, and $\phi'(\alpha_i)$. (This cubic function always exists and is unique; see, for example, Bulirsch and Stoer [29, p. 52].) The minimizer of this cubic in $[a, b]$ is either at one of the endpoints or else in the interior, in which case it is given by

$$\alpha_{i+1} = \alpha_i - (\alpha_i - \alpha_{i-1})\left[\frac{\phi'(\alpha_i) + d_2 - d_1}{\phi'(\alpha_i) - \phi'(\alpha_{i-1}) + 2d_2}\right], \tag{3.43}$$

with

$$\begin{aligned} d_1 &= \phi'(\alpha_{i-1}) + \phi'(\alpha_i) - 3\frac{\phi(\alpha_{i-1}) - \phi(\alpha_i)}{\alpha_{i-1} - \alpha_i}, \\ d_2 &= \left[d_1^2 - \phi'(\alpha_{i-1})\phi'(\alpha_i)\right]^{1/2}. \end{aligned}$$

The interpolation process can be repeated by discarding the data at one of the step lengths α_{i-1} or α_i and replacing it by $\phi(\alpha_{i+1})$ and $\phi'(\alpha_{i+1})$. The decision on which of α_{i-1} and α_i should be kept and which discarded depends on the specific conditions used to terminate the line search; we discuss this issue further below in the context of the Wolfe conditions. Cubic interpolation is a powerful strategy, since it can produce a quadratic rate of convergence of the iteration (3.43) to the minimizing value of α.

THE INITIAL STEP LENGTH

For Newton and quasi-Newton methods the step $\alpha_0 = 1$ should always be used as the initial trial step length. This choice ensures that unit step lengths are taken whenever they satisfy the termination conditions and allows the rapid rate-of-convergence properties of these methods to take effect.

For methods that do not produce well-scaled search directions, such as the steepest descent and conjugate gradient methods, it is important to use current information about the problem and the algorithm to make the initial guess. A popular strategy is to assume that the first-order change in the function at iterate x_k will be the same as that obtained at the previous step. In other words, we choose the initial guess α_0 so that $\alpha_0 \nabla f_k^T p_k = \alpha_{k-1} \nabla f_{k-1}^T p_{k-1}$. We therefore have

$$\alpha_0 = \alpha_{k-1} \frac{\nabla f_{k-1}^T p_{k-1}}{\nabla f_k^T p_k}.$$

Another useful strategy is to interpolate a quadratic to the data $f(x_{k-1})$, $f(x_k)$, and $\phi'(0) = \nabla f_k^T p_k$ and to define α_0 to be its minimizer. This strategy yields

$$\alpha_0 = \frac{2(f_k - f_{k-1})}{\phi'(0)}. \tag{3.44}$$

It can be shown that if $x_k \to x^*$ superlinearly, then the ratio in this expression converges to 1. If we adjust the choice (3.44) by setting

$$\alpha_0 \leftarrow \min(1, 1.01\alpha_0),$$

we find that the unit step length $\alpha_0 = 1$ will eventually always be tried and accepted, and the superlinear convergence properties of Newton and quasi-Newton methods will be observed.

A LINE SEARCH ALGORITHM FOR THE WOLFE CONDITIONS

The Wolfe (or strong Wolfe) conditions are among the most widely applicable and useful termination conditions. We now describe in some detail a one-dimensional search procedure that is guaranteed to find a step length satisfying the *strong* Wolfe conditions (3.7)

for any parameters c_1 and c_2 satisfying $0 < c_1 < c_2 < 1$. As before, we assume that p is a descent direction and that f is bounded below along the direction p.

The algorithm has two stages. This first stage begins with a trial estimate α_1, and keeps increasing it until it finds either an acceptable step length or an interval that brackets the desired step lengths. In the latter case, the second stage is invoked by calling a function called **zoom** (Algorithm 3.3 below), which successively decreases the size of the interval until an acceptable step length is identified.

A formal specification of the line search algorithm follows. We refer to (3.7a) as the *sufficient decrease condition* and to (3.7b) as the *curvature condition*. The parameter $\alpha_{\max}$ is a user-supplied bound on the maximum step length allowed. The line search algorithm terminates with α_* set to a step length that satisfies the strong Wolfe conditions.

Algorithm 3.2 (Line Search Algorithm).
Set $\alpha_0 \leftarrow 0$, choose $\alpha_1 > 0$ and $\alpha_{\max}$;
$i \leftarrow 1$;
repeat
  Evaluate $\phi(\alpha_i)$;
  if $\phi(\alpha_i) > \phi(0) + c_1\alpha_i\phi'(0)$ or $[\phi(\alpha_i) \geq \phi(\alpha_{i-1})$ and $i > 1]$
    $\alpha_* \leftarrow$**zoom**(α_{i-1}, α_i) and **stop**;
  Evaluate $\phi'(\alpha_i)$;
  if $|\phi'(\alpha_i)| \leq -c_2\phi'(0)$
    set $\alpha_* \leftarrow \alpha_i$ and **stop**;
  if $\phi'(\alpha_i) \geq 0$
    set $\alpha_* \leftarrow$**zoom**(α_i, α_{i-1}) and **stop**;
  Choose $\alpha_{i+1} \in (\alpha_i, \alpha_{\max})$
  $i \leftarrow i + 1$;
end (repeat)

Note that the sequence of trial step lengths $\{\alpha_i\}$ is monotonically increasing, but that the order of the arguments supplied to the **zoom** function may vary. The procedure uses the knowledge that the interval (α_{i-1}, α_i) contains step lengths satisfying the strong Wolfe conditions if one of the following three conditions is satisfied:

(i) α_i violates the sufficient decrease condition;

(ii) $\phi(\alpha_i) \geq \phi(\alpha_{i-1})$;

(iii) $\phi'(\alpha_i) \geq 0$.

The last step of the algorithm performs extrapolation to find the next trial value α_{i+1}. To implement this step we can use approaches like the interpolation procedures above, or we can simply set α_{i+1} to some constant multiple of α_i. Whichever strategy we use, it is important

that the successive steps increase quickly enough to reach the upper limit $\alpha_{\max}$ in a finite number of iterations.

We now specify the function **zoom**, which requires a little explanation. The order of its input arguments is such that each call has the form **zoom**$(\alpha_{\text{lo}}, \alpha_{\text{hi}})$, where

(a) the interval bounded by α_{lo} and α_{hi} contains step lengths that satisfy the strong Wolfe conditions;

(b) α_{lo} is, among all step lengths generated so far and satisfying the sufficient decrease condition, the one giving the smallest function value; and

(c) α_{hi} is chosen so that $\phi'(\alpha_{\text{lo}})(\alpha_{\text{hi}} - \alpha_{\text{lo}}) < 0$.

Each iteration of **zoom** generates an iterate α_j between α_{lo} and α_{hi}, and then replaces one of these endpoints by α_j in such a way that the properties (a), (b), and (c) continue to hold.

Algorithm 3.3 (zoom).

repeat
  Interpolate (using quadratic, cubic, or bisection) to find
    a trial step length α_j between α_{lo} and α_{hi};
  Evaluate $\phi(\alpha_j)$;
  if $\phi(\alpha_j) > \phi(0) + c_1\alpha_j\phi'(0)$ or $\phi(\alpha_j) \geq \phi(\alpha_{\text{lo}})$
    $\alpha_{\text{hi}} \leftarrow \alpha_j$;
  else
    Evaluate $\phi'(\alpha_j)$;
    if $|\phi'(\alpha_j)| \leq -c_2\phi'(0)$
      Set $\alpha_* \leftarrow \alpha_j$ and **stop**;
    if $\phi'(\alpha_j)(\alpha_{\text{hi}} - \alpha_{\text{lo}}) \geq 0$
      $\alpha_{\text{hi}} \leftarrow \alpha_{\text{lo}}$;
    $\alpha_{\text{lo}} \leftarrow \alpha_j$;
end (**repeat**)

If the new estimate α_j happens to satisfy the strong Wolfe conditions, then **zoom** has served its purpose of identifying such a point, so it terminates with $\alpha_* = \alpha_j$. Otherwise, if α_j satisfies the sufficient decrease condition and has a lower function value than x_{lo}, then we set $\alpha_{\text{lo}} \leftarrow \alpha_j$ to maintain condition (b). If this results in a violation of condition (c), we remedy the situation by setting α_{hi} to the old value of α_{lo}. The reader should sketch some graphs to illustrate the workings of **zoom**!

As mentioned earlier, the interpolation step that determines α_j should be safeguarded to ensure that the new step length is not too close to the endpoints of the interval. Practical line search algorithms also make use of the properties of the interpolating polynomials to make educated guesses of where the next step length should lie; see [27, 172]. A problem that can arise in the implementation is that as the optimization algorithm approaches the

solution, two consecutive function values $f(x_k)$ and $f(x_{k-1})$ may be indistinguishable in finite-precision arithmetic. Therefore, the line search must include a stopping test if it cannot attain a lower function value after a certain number (typically, ten) of trial step lengths. Some procedures also stop if the relative change in x is close to machine accuracy, or to some user-specified threshold.

A line search algorithm that incorporates all these features is difficult to code. We advocate the use of one of the several good software implementations available in the public domain. See Dennis and Schnabel [69], Lemaréchal [149], Fletcher [83], and in particular Moré and Thuente [172].

One may ask how much more expensive it is to require the strong Wolfe conditions instead of the regular Wolfe conditions. Our experience suggests that for a "loose" line search (with parameters such as $c_1 = 10^{-4}$ and $c_2 = 0.9$), both strategies require a similar amount of work. The strong Wolfe conditions have the advantage that by decreasing c_2 we can directly control the quality of the search by forcing the accepted value of α to lie closer to a local minimum. This feature is important in steepest descent or nonlinear conjugate gradient methods, and therefore a step selection routine that enforces the strong Wolfe conditions is of wider applicability.

NOTES AND REFERENCES

For an extensive discussion of line search termination conditions see Ortega and Rheinboldt [185]. Akaike [2] presents a probabilistic analysis of the steepest descent method with exact line searches on quadratic functions. He shows that when $n > 2$, the worst-case bound (3.29) can be expected to hold for most starting points. The case where $n = 2$ can be studied in closed form; see Bazaraa, Sherali, and Shetty [7].

Some line search methods (see Goldfarb [113] and Moré and Sorensen [169]) compute a direction of negative curvature, whenever it exists, to prevent the iteration from converging to nonminimizing stationary points. A direction of negative curvature p_- is one that satisfies $p_-^T \nabla^2 f(x_k) p_- < 0$. These algorithms generate a search direction by combining p_- with the steepest descent direction $-\nabla f$, and often perform a curvilinear backtracking line search. It is difficult to determine the relative contributions of the steepest descent and negative curvature directions, and due to this, this approach fell out of favor after the introduction of trust-region methods.

For a discussion on the rate of convergence of the coordinate descent method and for more references about this method see Luenberger [152].

Derivative-free line search algorithms include golden section and Fibonacci search. They share some of the features with the line search method given in this chapter. They typically store three trial points that determine an interval containing a one-dimensional minimizer. Golden section and Fibonacci differ in the way in which the trial step lengths are generated; see, for example, [58, 27].

Our discussion of interpolation follows Dennis and Schnabel [69], and the algorithm for finding a step length satisfying the strong Wolfe conditions can be found in Fletcher [83].